

## My Project

Generated by Doxygen 1.9.8



|                                                 |           |
|-------------------------------------------------|-----------|
| <b>1 SGBD ITP</b>                               | <b>1</b>  |
| 1.1 Instalação                                  | 2         |
| 1.2 Detalhes da implementação                   | 2         |
| 1.3 O que poderia ser feito de forma diferente? | 2         |
| 1.4 Autora                                      | 2         |
| <b>2 Data Structure Index</b>                   | <b>3</b>  |
| 2.1 Data Structures                             | 3         |
| <b>3 File Index</b>                             | <b>5</b>  |
| 3.1 File List                                   | 5         |
| <b>4 Data Structure Documentation</b>           | <b>7</b>  |
| 4.1 ColunaDaTabela Struct Reference             | 7         |
| 4.1.1 Detailed Description                      | 7         |
| 4.1.2 Field Documentation                       | 7         |
| 4.1.2.1 dados                                   | 7         |
| 4.1.2.2 nome                                    | 7         |
| 4.1.2.3 quantidadeDeLinhas                      | 7         |
| 4.1.2.4 tipo                                    | 7         |
| 4.2 Tabela Struct Reference                     | 8         |
| 4.2.1 Detailed Description                      | 8         |
| 4.2.2 Field Documentation                       | 8         |
| 4.2.2.1 colunas                                 | 8         |
| 4.2.2.2 nome                                    | 8         |
| 4.2.2.3 quantidadeDeColunas                     | 8         |
| 4.2.2.4 quantidadeDeLinhas                      | 8         |
| 4.3 Texto Struct Reference                      | 8         |
| 4.3.1 Detailed Description                      | 8         |
| 4.3.2 Field Documentation                       | 9         |
| 4.3.2.1 capacidade                              | 9         |
| 4.3.2.2 tamanho                                 | 9         |
| 4.3.2.3 vetor                                   | 9         |
| <b>5 File Documentation</b>                     | <b>11</b> |
| 5.1 Dados.c File Reference                      | 11        |
| 5.1.1 Function Documentation                    | 11        |
| 5.1.1.1 ehDigito()                              | 11        |
| 5.1.1.2 ehMaiuscula()                           | 11        |
| 5.1.1.3 ehMinuscula()                           | 12        |
| 5.1.1.4 ehSimbolo()                             | 12        |
| 5.1.1.5 texto_gravar()                          | 12        |
| 5.1.1.6 texto_lerTerminalEmLoop()               | 12        |
| 5.1.1.7 texto_lerTipoEmLoop()                   | 13        |

|                                         |    |
|-----------------------------------------|----|
| 5.1.1.8 texto_lerValorComTipo()         | 13 |
| 5.1.1.9 texto_lerValorDoTipoString()    | 13 |
| 5.1.1.10 texto_lerValorPositivoEmLoop() | 13 |
| 5.1.1.11 texto_recuperar()              | 14 |
| 5.1.1.12 verificaNomeDaTabelaValido()   | 14 |
| 5.2 Dados.h File Reference              | 14 |
| 5.2.1 Function Documentation            | 15 |
| 5.2.1.1 texto_gravar()                  | 15 |
| 5.2.1.2 texto_lerTerminalEmLoop()       | 15 |
| 5.2.1.3 texto_lerTipoEmLoop()           | 15 |
| 5.2.1.4 texto_lerValorComTipo()         | 15 |
| 5.2.1.5 texto_lerValorPositivoEmLoop()  | 16 |
| 5.2.1.6 texto_recuperar()               | 16 |
| 5.3 Dados.h                             | 16 |
| 5.4 Pesquisa.c File Reference           | 17 |
| 5.4.1 Enumeration Type Documentation    | 17 |
| 5.4.1.1 Regra                           | 17 |
| 5.4.2 Function Documentation            | 17 |
| 5.4.2.1 compararProximo()               | 17 |
| 5.4.2.2 compararValores()               | 18 |
| 5.4.2.3 deveExibir()                    | 18 |
| 5.4.2.4 pesquisa_exibir()               | 18 |
| 5.4.2.5 pesquisar()                     | 19 |
| 5.5 Pesquisa.h File Reference           | 19 |
| 5.5.1 Function Documentation            | 19 |
| 5.5.1.1 pesquisar()                     | 19 |
| 5.6 Pesquisa.h                          | 19 |
| 5.7 principal.c File Reference          | 20 |
| 5.7.1 Enumeration Type Documentation    | 20 |
| 5.7.1.1 Opcoes                          | 20 |
| 5.7.2 Function Documentation            | 21 |
| 5.7.2.1 adicionarDados()                | 21 |
| 5.7.2.2 apagarTabela()                  | 21 |
| 5.7.2.3 apagarTupla()                   | 21 |
| 5.7.2.4 criarTabela()                   | 21 |
| 5.7.2.5 exibirDados()                   | 21 |
| 5.7.2.6 leituraDaChavePrimaria()        | 21 |
| 5.7.2.7 leituraDeDados()                | 21 |
| 5.7.2.8 main()                          | 22 |
| 5.7.2.9 menu()                          | 22 |
| 5.7.2.10 pesquisarValores()             | 22 |
| 5.7.2.11 verificarExistenciaDaChave()   | 22 |

|                                        |    |
|----------------------------------------|----|
| 5.7.2.12 verificarNomesIguais()        | 22 |
| 5.8 README.md File Reference           | 23 |
| 5.9 Tabela.c File Reference            | 23 |
| 5.9.1 Function Documentation           | 23 |
| 5.9.1.1 coluna_buscarValor()           | 23 |
| 5.9.1.2 tabela_adicionarLinha()        | 23 |
| 5.9.1.3 tabela_criar()                 | 24 |
| 5.9.1.4 tabela_exibirCabecalho()       | 24 |
| 5.9.1.5 tabela_exibirDados()           | 24 |
| 5.9.1.6 tabela_gravar()                | 24 |
| 5.9.1.7 tabela_liberar()               | 25 |
| 5.9.1.8 tabela_recuperar()             | 25 |
| 5.9.1.9 tabela_removeChave()           | 25 |
| 5.10 Tabela.h File Reference           | 26 |
| 5.10.1 Function Documentation          | 26 |
| 5.10.1.1 coluna_buscarValor()          | 26 |
| 5.10.1.2 tabela_adicionarLinha()       | 26 |
| 5.10.1.3 tabela_criar()                | 27 |
| 5.10.1.4 tabela_exibirCabecalho()      | 27 |
| 5.10.1.5 tabela_exibirDados()          | 27 |
| 5.10.1.6 tabela_gravar()               | 28 |
| 5.10.1.7 tabela_liberar()              | 28 |
| 5.10.1.8 tabela_recuperar()            | 28 |
| 5.10.1.9 tabela_removeChave()          | 28 |
| 5.11 Tabela.h                          | 29 |
| 5.12 TabelaGeral.c File Reference      | 29 |
| 5.12.1 Macro Definition Documentation  | 30 |
| 5.12.1.1 NOME_DA_TABELA_GERAL          | 30 |
| 5.12.2 Function Documentation          | 30 |
| 5.12.2.1 tabelaGeral_adicionarTabela() | 30 |
| 5.12.2.2 tabelaGeral_apagar()          | 30 |
| 5.12.2.3 tabelaGeral_contemTabela()    | 30 |
| 5.12.2.4 tabelaGeral_gravarDados()     | 30 |
| 5.12.2.5 tabelaGeral_limpar()          | 31 |
| 5.12.2.6 tabelaGeral_listar()          | 31 |
| 5.12.2.7 tabelaGeral_recuperarDados()  | 31 |
| 5.13 TabelaGeral.h File Reference      | 31 |
| 5.13.1 Function Documentation          | 32 |
| 5.13.1.1 tabelaGeral_adicionarTabela() | 32 |
| 5.13.1.2 tabelaGeral_apagar()          | 32 |
| 5.13.1.3 tabelaGeral_contemTabela()    | 32 |
| 5.13.1.4 tabelaGeral_limpar()          | 32 |

|                               |           |
|-------------------------------|-----------|
| 5.13.1.5 tabelaGeral_listar() | 32        |
| 5.14 TabelaGeral.h            | 33        |
| <b>Index</b>                  | <b>35</b> |

# Chapter 1

## SGBD ITP

O projeto consiste em implementar um Sistema de Gerenciamento de Banco de Dados (SGBD) simplificado baseado no modelo relacional. Não é o objetivo desse projeto criar um SGBD para ser utilizado em produção por sistemas de informação, mas “apenas” explorar os conceitos abordados na disciplina ITP. O SGBD ITP deverá ser capaz de realizar as seguintes operações:

1. Criar uma tabela;
2. Listar todas as tabelas;
3. Criar uma nova linha na tabela;
4. Listar todos os dados da tabela;
5. Pesquisar valor em uma tabela;
6. Apagar uma tupla ou linha de uma tabela;
7. Apagar uma tabela;
8. Apagar todas as tabelas (funcionalidade adicional).

O SGBD ITP implementado cria um arquivo para cada tabela, além de criar um arquivo extra contendo a quantidade de tabelas e seus nomes para auxiliar, quando necessário, na implementação das funções.

O arquivo que contém a quantidade de tabelas e seus nomes se chama [Tabela](#) Geral.data e está organizado da seguinte forma: o primeiro dado corresponde à quantidade de tabelas existentes; o segundo dado corresponde ao tamanho da string seguido pelo nome de uma tabela separados por espaço; os próximos dados são equivalentes ao segundo.

Cada arquivo das tabelas está organizado da seguinte forma: Tamanho da string do nome da tabela seguido pelo seu nome e separados por espaço; quantidade de colunas; quantidade de linhas; tipo de dado da coluna; tamanho da string do nome da coluna e seu nome, separados por espaço; tamanho da string do dado e o dado em si, separados por espaço; e repete esse formato - tipo, tam nome, tam dado - para as outras colunas.

O código fonte foi dividido em 4 módulos além do principal, chamados: [Dados.h](#), [Pesquisa.h](#), [Tabela.h](#), [TabelaGeral.h](#) e [principal.c](#). No módulo Dados, estão organizadas as funções responsáveis pelas leituras de dados e gravações das informações obtidas nos arquivos. No módulo Pesquisa, estão organizadas as funções responsáveis pela pesquisa de um valor em uma tabela. No módulo [Tabela](#), estão organizadas as funções responsáveis pelas operações relacionadas às tabelas, como a criação de uma tabela, a adição ou remoção de uma linha, a exibição de seus dados... No módulo TabelaGeral, estão organizadas as funções responsáveis pelas operações relacionadas ao arquivo 'tabela geral.data', como sua criação, a atualização de suas informações, a exibição dos nomes de todas as tabelas, a remoção de um arquivo... No módulo principal, está implementada a função principal, além de outras auxiliares.

O código fonte está documentado no arquivo Doc.pdf.

## 1.1 Instalação

O código fonte está escrito na linguagem C, sendo necessário apenas de um compilador para a referida linguagem voltado a sua plataforma. Por exemplo, para o compilador GCC, pode executar a seguinte linha de comando no diretório que se encontra o código fonte:

```
gcc -o main -Wno-unused-result principal.c Dados.c Pesquisa.c Tabela.c TabelaGeral.c
```

Versão do compilador GCC: gcc version 6.3.0 (MinGW.org GCC-6.3.0-1) Link para download: <https://www.mingw-w64.org/> Importante ressaltar que não é necessária a instalação de nenhuma biblioteca externa.

## 1.2 Detalhes da implementação

Todas as funções solicitadas foram implementadas, além de uma função extra (Apagar todas as tabelas do código fonte) e a opção de sair do menu.

## 1.3 O que poderia ser feito de forma diferente?

Poderia ter sido criado um único arquivo para todas as tabelas. Porém, a implementação se tornou muito complexa para a autora, que decidiu reiniciar o projeto utilizando a ideia atual. Outra coisa que poderia ter sido diferente era ter salvo os dados nos arquivos de modo binário e não em caractere, reduzindo assim o tamanho do arquivo. Também poderia ter sido implementada uma função extra de remover uma coluna na tabela.

## 1.4 Autora

A autora do projeto se chama Camila Braga de Oliveira.



## Chapter 2

# Data Structure Index

### 2.1 Data Structures

Here are the data structures with brief descriptions:

|                                |   |
|--------------------------------|---|
| <a href="#">ColunaDaTabela</a> | 7 |
| <a href="#">Tabela</a>         | 8 |
| <a href="#">Texto</a>          | 8 |



## Chapter 3

# File Index

### 3.1 File List

Here is a list of all files with brief descriptions:

|               |    |
|---------------|----|
| Dados.c       | 11 |
| Dados.h       | 14 |
| Pesquisa.c    | 17 |
| Pesquisa.h    | 19 |
| principal.c   | 20 |
| Tabela.c      | 23 |
| Tabela.h      | 26 |
| TabelaGeral.c | 29 |
| TabelaGeral.h | 31 |



# Chapter 4

## Data Structure Documentation

### 4.1 ColunaDaTabela Struct Reference

```
#include <Tabela.h>
```

#### Data Fields

- [Texto](#) nome
- char [tipo](#)
- int [quantidadeDeLinhas](#)
- [Texto](#) \* [dados](#)

#### 4.1.1 Detailed Description

Representa uma coluna de uma tabela.

Representa uma coluna de uma tabela. Uma coluna tem um nome, que é representado pela estrutura de [Texto](#), um tipo associado, que pode ser 'i', 'f', 'd', 'c' ou 's', a quantidade de linhas presentes na coluna e um vetor para armazenar os dados da coluna.

#### 4.1.2 Field Documentation

##### 4.1.2.1 dados

[Texto](#)\* dados

Os dados da coluna

##### 4.1.2.2 nome

[Texto](#) nome

Nome da coluna

##### 4.1.2.3 quantidadeDeLinhas

int quantidadeDeLinhas

quantidade de linhas.

##### 4.1.2.4 tipo

char tipo

o tipo de dado associado à coluna.

The documentation for this struct was generated from the following file:

- [Tabela.h](#)

## 4.2 Tabela Struct Reference

```
#include <Tabela.h>
```

### Data Fields

- [Texto](#) nome
- int [quantidadeDeColunas](#)
- int [quantidadeDeLinhas](#)
- [ColunaDaTabela](#) \* [colunas](#)

### 4.2.1 Detailed Description

Representa uma tabela.

Representa uma tabela de dados do sistema. Uma tabela possui um nome, que é representado pela estrutura de [Texto](#), um vetor para armazenar os dados de cada uma das colunas e uma variável para indicar a quantidade de colunas presentes na tabela. Os detalhes de cada coluna serão armazenados pela estrutura [ColunaDaTabela](#). Além disso, possui um campo para indicar a quantidade linhas das tabelas, ou seja, a quantidade de dados em cada coluna.

### 4.2.2 Field Documentation

#### 4.2.2.1 [colunas](#)

[ColunaDaTabela](#)\* [colunas](#)  
dados das colunas da tabela

#### 4.2.2.2 [nome](#)

[Texto](#) nome  
Nome da tabela

#### 4.2.2.3 [quantidadeDeColunas](#)

int [quantidadeDeColunas](#)  
Quantidade de colunas da tabela

#### 4.2.2.4 [quantidadeDeLinhas](#)

int [quantidadeDeLinhas](#)  
Quantidade de linhas da tabela

The documentation for this struct was generated from the following file:

- [Tabela.h](#)

## 4.3 Texto Struct Reference

```
#include <Dados.h>
```

### Data Fields

- int [capacidade](#)
- int [tamanho](#)
- char \* [vetor](#)

### 4.3.1 Detailed Description

Representa uma string de caracteres. Tem como objetivo armazenar informações de uma string , como seu tamanho, para facilitar as manipulações com arquivos.

## 4.3.2 Field Documentation

### 4.3.2.1 capacidade

`int capacidade`  
quantidade de posições alocadas para o vetor

### 4.3.2.2 tamanho

`int tamanho`  
tamanho do texto sem o `\0`

### 4.3.2.3 vetor

`char* vetor`  
vetor de caracteres do texto

The documentation for this struct was generated from the following file:

- [Dados.h](#)





# Chapter 5

## File Documentation

### 5.1 Dados.c File Reference

```
#include <string.h>
#include "Dados.h"
```

#### Functions

- int [ehMinuscula](#) (char c)
- int [ehMaiuscula](#) (char c)
- int [ehDigito](#) (char c)
- int [ehSimbolo](#) (char c)
- int [verificaNomeDaTabelaValido](#) (char \*nome, int tamanho)
- void [texto\\_gravar](#) ([Texto](#) texto, FILE \*arquivo)
- [Texto](#) [texto\\_recuperar](#) (FILE \*arquivo)
- [Texto](#) [texto\\_lerTerminalEmLoop](#) (int capacidade)
- int [texto\\_lerValorPositivoEmLoop](#) ()
- char [texto\\_lerTipoEmLoop](#) ()
- [Texto](#) [texto\\_lerValorDoTipoString](#) ()
- [Texto](#) [texto\\_lerValorComTipo](#) (char tipo)

#### 5.1.1 Function Documentation

##### 5.1.1.1 ehDigito()

```
int ehDigito (
    char c )
```

Verifica se o caractere é um dígito.

#### Parameters

|          |                             |
|----------|-----------------------------|
| <i>c</i> | Caractere sendo verificado. |
|----------|-----------------------------|

#### Returns

1 se o caractere for um dígito e 0 em caso contrário.

##### 5.1.1.2 ehMaiuscula()

```
int ehMaiuscula (
    char c )
```

Verifica se o caractere é uma letra maiúscula.

**Parameters**

|          |                             |
|----------|-----------------------------|
| <i>c</i> | Caractere sendo verificado. |
|----------|-----------------------------|

**Returns**

1 se o caractere for uma letra maiúscula e 0 em caso contrário.

**5.1.1.3 ehMinuscula()**

```
int ehMinuscula (
    char c )
```

Verifica se o caractere é uma letra minúscula.

**Parameters**

|          |                             |
|----------|-----------------------------|
| <i>c</i> | Caractere sendo verificado. |
|----------|-----------------------------|

**Returns**

1 se o caractere for uma letra minúscula e 0 em caso contrário.

**5.1.1.4 ehSimbolo()**

```
int ehSimbolo (
    char c )
```

Verifica se o caractere é um símbolo.

**Parameters**

|          |                             |
|----------|-----------------------------|
| <i>c</i> | Caractere sendo verificado. |
|----------|-----------------------------|

**Returns**

1 se o caractere for um símbolo e 0 em caso contrário.

**5.1.1.5 texto\_gravar()**

```
void texto_gravar (
    Texto texto,
    FILE * arquivo )
```

Grava os dados do texto em um arquivo aberto em modo de escrita de caracteres.

Os dados são salvos em dois campos no seguinte formato: "%d %s\n", correspondendo ao tamanho do texto e ao texto em si.

**Parameters**

|                |                                                                                               |
|----------------|-----------------------------------------------------------------------------------------------|
| <i>texto</i>   | texto a ser gravado.                                                                          |
| <i>arquivo</i> | Arquivo onde serão gravados os dados. Ele deve estar aberto em modo de escrita de caracteres. |

**5.1.1.6 texto\_lerTerminalEmLoop()**

```
Texto texto_lerTerminalEmLoop (
    int capacidade )
```

Faz a leitura de um texto e verifica se ele é válido. Caso seja válido, retorna uma estrutura com os dados lidos e, em caso contrário, solicita ao usuário que digite outro texto.

Para ser válido, o texto digitado deve conter apenas letras, numeros, ' ', '\_', '-' ou '.' além de iniciar apenas por letra ou dígito.

#### Parameters

|                   |                                         |
|-------------------|-----------------------------------------|
| <i>capacidade</i> | Espaço do vetor para leitura dos dados. |
|-------------------|-----------------------------------------|

#### Returns

Uma estrutura de texto com os dados lidos.

#### 5.1.1.7 texto\_lerTipoEmLoop()

```
char texto_lerTipoEmLoop ( )
```

Faz a leitura de um caractere e verifica se o dado digitado é válido. Para ser válido, o caractere deve ser apenas 'i', 'f', 'd', 'c' ou 's'. Em caso de sucesso, retorna o dado. Caso contrário, solicita ao usuário um dado novo.

#### Returns

Um caractere contendo o dado lido.

#### 5.1.1.8 texto\_lerValorComTipo()

```
Texto texto_lerValorComTipo (
    char tipo )
```

Faz a leitura e conversão de dados de acordo com a opção escolhida pelo usuário.

Recebe como argumento o tipo de dado da coluna escolhida pelo usuário, faz a leitura dos dados, os converte em string e depois armazena os dados convertidos no vetor de string. Observação: Para a variável "tipo" igual a 'f' ou 'd', a leitura e tratamento serão as mesmas.

#### Parameters

|             |                                                            |
|-------------|------------------------------------------------------------|
| <i>tipo</i> | Tipo de dado da coluna escolhida previamente pelo usuário. |
|-------------|------------------------------------------------------------|

#### Returns

Uma estrutura de texto com os dados lidos.

#### 5.1.1.9 texto\_lerValorDoTipoString()

```
Texto texto_lerValorDoTipoString ( )
```

Faz a leitura e tratamento de um texto, retornando uma estrutura com os dados desse texto.

Realiza a leitura caractere a caractere, enquanto não for digitado um '\n' ou um '\0'. Caso seja digitado '\n', realiza o tratamento que consiste em sobrepor esse último caractere digitado por '\0'.

#### Returns

Uma estrutura de texto com os dados lidos.

#### 5.1.1.10 texto\_lerValorPositivoEmLoop()

```
int texto_lerValorPositivoEmLoop ( )
```

Realiza a leitura de um inteiro positivo e verifica se é um dado válido. Caso seja válido, retorna o dado lido e, em caso contrário, solicita ao usuário um novo dado.

Para ser válido, o número deve ser um inteiro não negativo.

**Returns**

Um inteiro contendo o dado lido.

**5.1.1.11 texto\_recuperar()**

```
Texto texto_recuperar (
    FILE * arquivo )
```

Recupera os dados de um texto de um arquivo aberto em modo de leitura de caracteres. Assume que os dados foram salvos no formato da função

**See also**

[texto\\_gravar](#).

**Parameters**

|                |                                                                               |
|----------------|-------------------------------------------------------------------------------|
| <i>arquivo</i> | Arquivo que será lido. Deve estar aberto no formato de leitura de caracteres. |
|----------------|-------------------------------------------------------------------------------|

**Returns**

Uma estrutura de [Texto](#) com os dados lidos.

**5.1.1.12 verificaNomeDaTabelaValido()**

```
int verificaNomeDaTabelaValido (
    char * nome,
    int tamanho )
```

Verifica se um nome é válido.

Os nomes das tabelas possuem restrições quanto aos caracteres. Para serem válidos, os nomes devem ser compostos por letras, dígitos, símbolos '-', '\_', '.' ou espaço, além de não poderem ser vazios nem começarem por espaço ou símbolo.

**Parameters**

|                |                                      |
|----------------|--------------------------------------|
| <i>nome</i>    | Ponteiro para nome a ser verificado. |
| <i>tamanho</i> | Quantidade de caracteres do nome.    |

**Returns**

1 caso o nome seja válido e 0 em caso contrário.

**5.2 Dados.h File Reference**

```
#include <stdio.h>
#include <stdlib.h>
```

**Data Structures**

- struct [Texto](#)

**Functions**

- void [texto\\_gravar](#) ([Texto](#) texto, FILE \*arquivo)
- [Texto](#) [texto\\_recuperar](#) (FILE \*arquivo)
- [Texto](#) [texto\\_lerTerminalEmLoop](#) (int capacidade)

- char [texto\\_lerTipoEmLoop](#) ()
- int [texto\\_lerValorPositivoEmLoop](#) ()
- [Texto texto\\_lerValorComTipo](#) (char tipo)

## 5.2.1 Function Documentation

### 5.2.1.1 [texto\\_gravar\(\)](#)

```
void texto_gravar (
    Texto texto,
    FILE * arquivo )
```

Grava os dados do texto em um arquivo aberto em modo de escrita de caracteres.

Os dados são salvos em dois campos no seguinte formato: "%d %s\n", correspondendo ao tamanho do texto e ao texto em si.

#### Parameters

|                |                                                                                               |
|----------------|-----------------------------------------------------------------------------------------------|
| <i>texto</i>   | texto a ser gravado.                                                                          |
| <i>arquivo</i> | Arquivo onde serão gravados os dados. Ele deve estar aberto em modo de escrita de caracteres. |

### 5.2.1.2 [texto\\_lerTerminalEmLoop\(\)](#)

```
Texto texto_lerTerminalEmLoop (
    int capacidade )
```

Faz a leitura de um texto e verifica se ele é válido. Caso seja válido, retorna uma estrutura com os dados lidos e, em caso contrário, solicita ao usuário que digite outro texto.

Para ser válido, o texto digitado deve conter apenas letras, numeros, ',', '\_', '-' ou '.' além de iniciar apenas por letra ou dígito.

#### Parameters

|                   |                                         |
|-------------------|-----------------------------------------|
| <i>capacidade</i> | Espaço do vetor para leitura dos dados. |
|-------------------|-----------------------------------------|

#### Returns

Uma estrutura de texto com os dados lidos.

### 5.2.1.3 [texto\\_lerTipoEmLoop\(\)](#)

```
char texto_lerTipoEmLoop ( )
```

Faz a leitura de um caractere e verifica se o dado digitado é válido. Para ser válido, o caractere deve ser apenas 'i', 'f', 'd', 'c' ou 's'. Em caso de sucesso, retorna o dado. Caso contrário, solicita ao usuário um dado novo.

#### Returns

Um caractere contendo o dado lido.

### 5.2.1.4 [texto\\_lerValorComTipo\(\)](#)

```
Texto texto_lerValorComTipo (
    char tipo )
```

Faz a leitura e conversão de dados de acordo com a opção escolhida pelo usuário.

Recebe como argumento o tipo de dado da coluna escolhida pelo usuário, faz a leitura dos dados, os converte em string e depois armazena os dados convertidos no vetor de string. Observação: Para a variável "tipo" igual a 'f' ou 'd', a leitura e tratamento serão as mesmas.

## Parameters

|             |                                                            |
|-------------|------------------------------------------------------------|
| <i>tipo</i> | Tipo de dado da coluna escolhida previamente pelo usuário. |
|-------------|------------------------------------------------------------|

## Returns

Uma estrutura de texto com os dados lidos.

5.2.1.5 `texto_lerValorPositivoEmLoop()`

```
int texto_lerValorPositivoEmLoop ( )
```

Realiza a leitura de um inteiro positivo e verifica se é um dado válido. Caso seja válido, retorna o dado lido e, em caso contrário, solícia ao usuário um novo dado.

Para ser válido, o número deve ser um inteiro não negativo.

## Returns

Um inteiro contendo o dado lido.

5.2.1.6 `texto_recuperar()`

```
Texto texto_recuperar (
    FILE * arquivo )
```

Recupera os dados de um texto de um arquivo aberto em modo de leitura de caracteres.

Assume que os dados foram salvos no formato da função

## See also

[texto\\_gravar](#).

## Parameters

|                |                                                                               |
|----------------|-------------------------------------------------------------------------------|
| <i>arquivo</i> | Arquivo que será lido. Deve estar aberto no formato de leitura de caracteres. |
|----------------|-------------------------------------------------------------------------------|

## Returns

Uma estrutura de [Texto](#) com os dados lidos.

5.3 `Dados.h`

[Go to the documentation of this file.](#)

```
00001
00002 #ifndef DADOS_H
00003 #define DADOS_H
00004
00005 #include <stdio.h>
00006 #include <stdlib.h>
00007
00012 typedef struct
00013 {
00014     int capacidade;
00015     int tamanho;
00016     char *vetor;
00017 } Texto;
00018
00019 void texto_gravar(Texto texto, FILE *arquivo);
00020 Texto texto_recuperar(FILE *arquivo);
00021 Texto texto_lerTerminalEmLoop(int capacidade);
00022 char texto_lerTipoEmLoop();
00023 int texto_lerValorPositivoEmLoop();
00024
00025 // NULL para dado faltando
00026 Texto texto_lerValorComTipo(char tipo);
00027
00028 #endif
```

## 5.4 Pesquisa.c File Reference

```
#include "Dados.h"
#include "Tabela.h"
#include "Pesquisa.h"
```

### Enumerations

- enum [Regra](#) {  
    [MAIOR](#) = 1 , [MAIOR\\_OU\\_IGUAL](#) = 2 , [IGUAL](#) = 3 , [MENOR\\_OU\\_IGUAL](#) = 4 ,  
    [MENOR](#) = 5 , [PROXIMO](#) = 6 }

### Functions

- int [compararValores](#) ([Texto](#) primeiro, [Texto](#) segundo, char tipo)
- int [compararProximo](#) ([Texto](#) primeiro, [Texto](#) segundo)
- int [deveExibir](#) ([Texto](#) valor, [Texto](#) comparar, [Regra](#) regra, char tipo)
- void [pesquisa\\_exibir](#) ([Tabela](#) tabela, int indice, [Texto](#) valor, [Regra](#) regra)
- void [pesquisar](#) ([Tabela](#) tabela, int indice)

### 5.4.1 Enumeration Type Documentation

#### 5.4.1.1 Regra

enum [Regra](#)

Representa as opções de escolha do usuário referente à pesquisa de valores.

#### Enumerator

|                                |                                                                          |
|--------------------------------|--------------------------------------------------------------------------|
| <a href="#">MAIOR</a>          | Para valores maiores do que o solicitado;                                |
| <a href="#">MAIOR_OU_IGUAL</a> | Para valores maiores ou iguais ao solicitado;                            |
| <a href="#">IGUAL</a>          | Para valores iguais ao solicitado;                                       |
| <a href="#">MENOR_OU_IGUAL</a> | Para valores menores ou iguais ao solicitado;                            |
| <a href="#">MENOR</a>          | Para valores menores do que o solicitado;                                |
| <a href="#">PROXIMO</a>        | Para valores próximos ao solicitado. Apenas para valores do tipo string. |

### 5.4.2 Function Documentation

#### 5.4.2.1 [compararProximo\(\)](#)

```
int compararProximo (  
    Texto primeiro,  
    Texto segundo )
```

Verifica se o segundo argumento é substring do primeiro. Retorna 1 em caso de sucesso e 0, caso contrário.

#### Parameters

|                 |                     |
|-----------------|---------------------|
| <i>primeiro</i> | Estrutura de texto. |
| <i>segundo</i>  | Estrutura de texto. |

**Returns**

1 caso de sucesso e 0, caso contrário.

**5.4.2.2 compararValores()**

```
int compararValores (
    Texto primeiro,
    Texto segundo,
    char tipo )
```

Compara valores de duas estruturas, convertendo-os nos tipos 'c', 'i', 'f', 'd', 's', retornando um valor negativo se o primeiro valor for menor do que o segundo, 0 se os valores forem iguais ou um valor positivo se o primeiro valor for maior do que o segundo.

Observação: Para o caso em que o tipo de dado for 'f', a implementação será idêntica ao caso em que o tipo de dado for 'd'.

**Parameters**

|                 |                                                       |
|-----------------|-------------------------------------------------------|
| <i>primeiro</i> | Estrutura contendo os dados do primeiro valor         |
| <i>segundo</i>  | Estrutura contendo os dados do segundo valor          |
| <i>tipo</i>     | Tipo de dado referente aos valores a serem comparados |

**Returns**

um valor negativo se o primeiro valor for menor do que o segundo, 0 se os valores forem iguais ou um valor positivo se o primeiro valor for maior do que o segundo.

**5.4.2.3 deveExibir()**

```
int deveExibir (
    Texto valor,
    Texto comparar,
    Regra regra,
    char tipo )
```

Verifica a linha satisfaz a regra informada.

**Parameters**

|                 |                                                              |
|-----------------|--------------------------------------------------------------|
| <i>valor</i>    | Estrutura de texto contendo o valor solicitado pelo usuário. |
| <i>comparar</i> | Estrutura de texto contendo os valores a serem comparados.   |
| <i>regra</i>    | Opção de pesquisa escolhida pelo usuário.                    |
| <i>tipo</i>     | Tipo de dado                                                 |

**Returns**

1 em caso de sucesso e 0, caso contrário.

**5.4.2.4 pesquisa\_exibir()**

```
void pesquisa_exibir (
    Tabela tabela,
    int indice,
    Texto valor,
    Regra regra )
```

Exibe na tela os resultados da pesquisa de valores.



## Parameters

|               |                                                                 |
|---------------|-----------------------------------------------------------------|
| <i>tabela</i> | Estrutura contendo os dados da tabela.                          |
| <i>indice</i> | posição da coluna                                               |
| <i>valor</i>  | Estrutura contendo os dados do valor a ser pesquisado           |
| <i>regra</i>  | Estrutura referente à opção de pesquisa escolhida pelo usuário. |

## 5.4.2.5 pesquisar()

```
void pesquisar (
    Tabela tabela,
    int indice )
```

Exibe na tela o menu com as opções de pesquisa, faz a leitura da escolha do usuário e do valor a ser comparado e exibe os resultados da pesquisa.

## Parameters

|               |                                       |
|---------------|---------------------------------------|
| <i>tabela</i> | Estrutura contendo os dados da tabela |
| <i>indice</i> | Posição da coluna                     |

## 5.5 Pesquisa.h File Reference

```
#include <string.h>
#include "Tabela.h"
```

## Functions

- void [pesquisar](#) (Tabela tabela, int indice)

## 5.5.1 Function Documentation

## 5.5.1.1 pesquisar()

```
void pesquisar (
    Tabela tabela,
    int indice )
```

Exibe na tela o menu com as opções de pesquisa, faz a leitura da escolha do usuário e do valor a ser comparado e exibe os resultados da pesquisa.

## Parameters

|               |                                       |
|---------------|---------------------------------------|
| <i>tabela</i> | Estrutura contendo os dados da tabela |
| <i>indice</i> | Posição da coluna                     |

## 5.6 Pesquisa.h

[Go to the documentation of this file.](#)

```
00001
00002 #ifndef PESQUISA_H
00003 #define PESQUISA_H
00004
00005 #include <string.h>
00006 #include "Tabela.h"
00007
```

```
00008 void pesquisar(Tabela tabela, int indice);
00009
00010 #endif
```

## 5.7 principal.c File Reference

```
#include <stdio.h>
#include <stdlib.h>
#include "Dados.h"
#include "Tabela.h"
#include "Pesquisa.h"
#include "TabelaGeral.h"
```

### Enumerations

- enum [Opcoes](#) {  
[CRIAR\\_TABELA](#) = 1 , [LISTAR\\_TABELA](#) = 2 , [CRIAR\\_LINHA](#) = 3 , [LISTAR\\_DADOS](#) = 4 ,  
[PESQUISAR\\_VALOR](#) = 5 , [APAGAR\\_TUPLA](#) = 6 , [APAGAR\\_TABELA](#) = 7 , [SAIR](#) = 8 ,  
[LIMPAR](#) = 0 }

### Functions

- void [menu](#) ()
- int [verificarNomesIguais](#) (int n, [Texto](#) linhaASerInserida[])
- void [criarTabela](#) ()
- void [exibirDados](#) ()
- [Texto](#) [leituraDaChavePrimaria](#) ()
- [Texto](#) \* [leituraDeDados](#) (int n, [ColunaDaTabela](#) colunas[])
- int [verificarExistenciaDaChave](#) ([Tabela](#) tabela, [Texto](#) linhaASerInserida[])
- void [adicionarDados](#) ()
- void [apagarTabela](#) ()
- void [apagarTupla](#) ()
- void [pesquisarValores](#) ()
- int [main](#) (void)

### 5.7.1 Enumeration Type Documentation

#### 5.7.1.1 Opcoes

enum [Opcoes](#)

Representa as opções de escolha do menu do programa.

#### Enumerator

|                                 |  |
|---------------------------------|--|
| <a href="#">CRIAR_TABELA</a>    |  |
| <a href="#">LISTAR_TABELA</a>   |  |
| <a href="#">CRIAR_LINHA</a>     |  |
| <a href="#">LISTAR_DADOS</a>    |  |
| <a href="#">PESQUISAR_VALOR</a> |  |
| <a href="#">APAGAR_TUPLA</a>    |  |
| <a href="#">APAGAR_TABELA</a>   |  |
| <a href="#">SAIR</a>            |  |
| <a href="#">LIMPAR</a>          |  |

## 5.7.2 Function Documentation

### 5.7.2.1 adicionarDados()

```
void adicionarDados ( )
```

Representa a opção do menu de adicionar linha em uma tabela.

Faz a leitura do nome da tabela e dos dados a serem adicionados e, em seguida, atualiza o arquivo da tabela, inserindo uma linha com os dados recentes.

### 5.7.2.2 apagarTabela()

```
void apagarTabela ( )
```

Representa a opção do menu de apagar uma tabela.

Faz a leitura do nome da tabela e a apaga do disco rígido, atualizando o arquivo 'tabela geral.data', que contém a quantidade de tabelas e seus nomes.

### 5.7.2.3 apagarTupla()

```
void apagarTupla ( )
```

Representa a opção do menu de apagar uma linha de uma tabela.

Faz a leitura do nome da tabela e da chave desejada e apaga sua linha, atualizando o arquivo da tabela.

### 5.7.2.4 criarTabela()

```
void criarTabela ( )
```

Representa a opção do menu de criar uma tabela.

Realiza a leitura do nome da tabela, verificando se este é válido, da quantidade de colunas da tabela, dos nomes das colunas (incluindo a chave primária) e seus tipos, criando um arquivo contendo essas informações e outro contendo a quantidade de tabelas e seus nomes (tabela geral.data).

### 5.7.2.5 exibirDados()

```
void exibirDados ( )
```

Representa a opção do menu de exibir os dados de uma tabela.

Faz a leitura do nome da tabela, abre seu arquivo correspondente e exibe na tela seu cabeçalho com seus dados.

### 5.7.2.6 leituraDaChavePrimaria()

```
Texto leituraDaChavePrimaria ( )
```

Faz a leitura da chave primária retornando uma estrutura contendo suas informações.

#### Returns

Estrutura com os dados lidos.

### 5.7.2.7 leituraDeDados()

```
Texto * leituraDeDados (
    int n,
    ColunaDaTabela colunas[] )
```

Faz as leituras dos dados da linha a ser inserida, retornando uma estrutura com os dados lidos.

Observação: Insere 'NULL' caso o usuário não queira adicionar um dado.

#### Parameters

|                |                                         |
|----------------|-----------------------------------------|
| <i>n</i>       | quantidade de colunas                   |
| <i>colunas</i> | Estrutura contendo os dados das colunas |

**Returns**

Uma estrutura com os dados lidos

**5.7.2.8 main()**

```
int main (
    void )
```

Função principal.

**Returns**

Resultado da execução do programa. 0 indica ausência de erros.

Instruções de acordo com a escolha do usuário.

**5.7.2.9 menu()**

```
void menu ( )
```

Exibe na tela as opções do menu.

**5.7.2.10 pesquisarValores()**

```
void pesquisarValores ( )
```

Representa a opção do menu de pesquisar um valor em uma tabela.

Faz a leitura do nome da tabela e exibe as colunas disponíveis. Depois, faz a leitura do valor a ser pesquisado, exibindo as opções de pesquisa para, posteriormente, exibir os resultados da pesquisa.

**5.7.2.11 verificarExistenciaDaChave()**

```
int verificarExistenciaDaChave (
    Tabela tabela,
    Texto linhaASerInserida[ ] )
```

Verifica se já existe uma chave com valor igual ao requerido, retornando 1 em caso de sucesso e 0, caso contrário.

**Parameters**

|                          |                                       |
|--------------------------|---------------------------------------|
| <i>tabela</i>            | Estrutura com os dados da tabela.     |
| <i>linhaASerInserida</i> | Estrutura com o dado a ser verificado |

**Returns**

1 em caso de sucesso e 0, caso contrário

**5.7.2.12 verificarNomesIguais()**

```
int verificarNomesIguais (
    int n,
    Texto linhaASerInserida[ ] )
```

Verifica se já existe um texto registrado igual ao requerido.

Recebe um vetor de textos e verifica se existe dois textos iguais nesse vetor. Retorna 0 se não existirem valores iguais ou, caso contrário, a posição de maior valor entre os dois textos comparados, ou seja, nunca retorna zero em caso verdadeiro.

**Parameters**

|                          |                                          |
|--------------------------|------------------------------------------|
| <i>n</i>                 | quantidade de textos a serem comparadas. |
| <i>linhaASerInserida</i> | vetor com todos os textos.               |

## Returns

retorna 0 se não existir dois textos iguais no vetor, caso contrário, retorna valor maior que zero.

## 5.8 README.md File Reference

## 5.9 Tabela.c File Reference

```
#include "Tabela.h"
```

## Functions

- [Tabela](#) [tabela\\_criar](#) ([Texto](#) nome, int quantidadeDeColunas, [Texto](#) nomesDasColunas[], char tiposDasColunas[])
- void [tabela\\_gravar](#) ([Tabela](#) tabela)
- [Tabela](#) [tabela\\_recuperar](#) ([Texto](#) nome)
- void [tabela\\_liberar](#) ([Tabela](#) tabela)
- void [tabela\\_exibirCabecalho](#) ([Tabela](#) tabela)
- void [tabela\\_exibirDados](#) ([Tabela](#) tabela)
- [Tabela](#) [tabela\\_adicionarLinha](#) ([Tabela](#) tabela, [Texto](#) dados[])
- int [coluna\\_buscarValor](#) ([ColunaDaTabela](#) coluna, [Texto](#) valor)
- int [tabela\\_removeChave](#) ([Tabela](#) tabela, [Texto](#) chave)

### 5.9.1 Function Documentation

#### 5.9.1.1 coluna\_buscarValor()

```
int coluna_buscarValor (
    ColunaDaTabela coluna,
    Texto valor )
```

Verifica se um dado existe nas colunas de uma tabela.

Recebe como parâmetros uma estrutura contendo os dados da coluna de uma tabela e outra contendo os dados de um valor. Depois, compara cada dado da coluna com esse valor, retornando a posição do dado valor na coluna em caso de sucesso (quando são iguais) e -1, caso contrário.

## Parameters

|               |                                                   |
|---------------|---------------------------------------------------|
| <i>coluna</i> | Estrutura de texto contendo os dados da tabela.   |
| <i>valor</i>  | Estrutura de texto contendo os dados de um valor. |

## Returns

-1 se o valor não estiver presente. Caso contrário, retorna a posição do valor na coluna.

#### 5.9.1.2 tabela\_adicionarLinha()

```
Tabela tabela_adicionarLinha (
    Tabela tabela,
    Texto dados[] )
```

Adiciona uma linha para cada coluna de uma tabela.

Recebe como parâmetro uma estrutura texto contendo os dados a serem adicionados e uma estrutura contendo os dados da tabela. Em seguida, atualiza a estrutura [Tabela](#) com os dados da estrutura [Texto](#) retornando-a no final.

## Parameters

|               |                                                                                  |
|---------------|----------------------------------------------------------------------------------|
| <i>tabela</i> | Estrutura de texto contendo os dados da tabela.                                  |
| <i>dados</i>  | Estrutura de string contendo os dados a serem adicionados nas colunas da tabela. |

**Returns**

Estrutura [Tabela](#) contendo os dados da tabela atualizados.

**5.9.1.3 tabela\_criar()**

```
Tabela tabela_criar (
    Texto nome,
    int quantidadeDeColunas,
    Texto nomesDasColunas[],
    char tiposDasColunas[] )
```

Cria uma tabela.

Recebe como argumentos o nome da tabela, a quantidade de colunas que a tabela terá, um vetor com os nomes das colunas e outro com os tipos de dados associados à elas. Retorna uma estrutura preenchida com essas informações.

**Parameters**

|                            |                                                                   |
|----------------------------|-------------------------------------------------------------------|
| <i>nome</i>                | Estrutura contendo os dados do nome da tabela.                    |
| <i>quantidadeDeColunas</i> | A quantidade de colunas que a tabela terá.                        |
| <i>nomesDasColunas</i>     | Vetor de string contendo os nomes das colunas.                    |
| <i>tiposDasColunas</i>     | Vetor de string contendo os tipos de dados associados às colunas. |

**Returns**

Estrutura preenchidas com as informações recebidas.

**5.9.1.4 tabela\_exibirCabecalho()**

```
void tabela_exibirCabecalho (
    Tabela tabela )
```

Exibe na tela o cabeçalho de uma tabela.

Recebe como parâmetro uma estrutura contendo o nome de uma tabela, a quantidade de linhas e colunas, os nomes e os tipos das colunas e exibe essas informações na tela.

**Parameters**

|               |                                                 |
|---------------|-------------------------------------------------|
| <i>tabela</i> | Estrutura de texto contendo os dados da tabela. |
|---------------|-------------------------------------------------|

**5.9.1.5 tabela\_exibirDados()**

```
void tabela_exibirDados (
    Tabela tabela )
```

Exibe na tela os dados das colunas de uma tabela.

Recebe como parâmetro uma estrutura contendo os dados de uma tabela e exibe na tela os dados de cada coluna.

**Parameters**

|               |                                                 |
|---------------|-------------------------------------------------|
| <i>tabela</i> | Estrutura de texto contendo os dados da tabela. |
|---------------|-------------------------------------------------|

**5.9.1.6 tabela\_gravar()**

```
void tabela_gravar (
    Tabela tabela )
```

Cria um arquivo referente a uma tabela.

Recebe como argumento uma estrutura contendo informações de uma tabela adquiridas previamente em outra função e cria um arquivo para essa tabela, repassando as informações contidas na estrutura para esse arquivo. Observação: o arquivo será nomeado pelo nome da tabela.

#### Parameters

|               |                                                  |
|---------------|--------------------------------------------------|
| <i>tabela</i> | Estrutura contendo as informações de uma tabela. |
|---------------|--------------------------------------------------|

#### 5.9.1.7 `tabela_liberar()`

```
void tabela_liberar (
    Tabela tabela )
```

Liberação da memória alocada dinamicamente para a estrutura tabela.

#### Parameters

|               |                                                 |
|---------------|-------------------------------------------------|
| <i>tabela</i> | Estrutura de texto contendo os dados da tabela. |
|---------------|-------------------------------------------------|

#### 5.9.1.8 `tabela_recuperar()`

```
Tabela tabela_recuperar (
    Texto nome )
```

Faz a leitura dos dados de um arquivo referente a uma tabela.

Recebe como argumento uma estrutura de texto contendo os dados do nome de uma tabela, abre o arquivo correspondente à essa tabela, em modo leitura e faz a leitura de seus dados, retornando uma estrutura contendo os dados lidos.

#### Parameters

|             |                                                         |
|-------------|---------------------------------------------------------|
| <i>nome</i> | Estrutura de texto contendo os dados do nome da tabela. |
|-------------|---------------------------------------------------------|

#### Returns

Uma estrutura contendo os dados lidos.

#### 5.9.1.9 `tabela_removeChave()`

```
int tabela_removeChave (
    Tabela tabela,
    Texto chave )
```

Remove uma linha (tupla) de uma tabela.

Recebe como parâmetros uma estrutura contendo os dados de uma tabela e outra contendo os dados de um valor correspondente à chave da linha a ser removida. Depois, localiza sua posição na coluna, caso ela exista e atualiza o arquivo da tabela com os dados restantes, excluindo-se os referentes à essa posição. Retorna 1 caso a remoção da linha seja realizada com sucesso e 0 caso a chave solicitada não exista.

#### Parameters

|               |                                                                                |
|---------------|--------------------------------------------------------------------------------|
| <i>tabela</i> | Estrutura de texto contendo os dados da tabela.                                |
| <i>chave</i>  | Estrutura de texto contendo os dados da chave referente à linha a ser removida |

**Returns**

1 caso a remoção da linha seja realizada com sucesso e 0 caso a chave não exista.

## 5.10 Tabela.h File Reference

```
#include <stdio.h>
#include <string.h>
#include "Dados.h"
```

**Data Structures**

- struct [ColunaDaTabela](#)
- struct [Tabela](#)

**Functions**

- [Tabela tabela\\_criar](#) ([Texto](#) nome, int quantidadeDeColunas, [Texto](#) nomeDasColunas[], char tipoDasColunas[])
- void [tabela\\_gravar](#) ([Tabela](#) tabela)
- [Tabela tabela\\_recuperar](#) ([Texto](#) nome)
- void [tabela\\_liberar](#) ([Tabela](#) tabela)
- void [tabela\\_exibirCabecalho](#) ([Tabela](#) tabela)
- void [tabela\\_exibirDados](#) ([Tabela](#) tabela)
- int [tabela\\_removeChave](#) ([Tabela](#) tabela, [Texto](#) chave)
- [Tabela tabela\\_adicionarLinha](#) ([Tabela](#) tabela, [Texto](#) dados[])
- int [coluna\\_buscarValor](#) ([ColunaDaTabela](#) coluna, [Texto](#) valor)

### 5.10.1 Function Documentation

#### 5.10.1.1 coluna\_buscarValor()

```
int coluna_buscarValor (
    ColunaDaTabela coluna,
    Texto valor )
```

Verifica se um dado existe nas colunas de uma tabela.

Recebe como parâmetros uma estrutura contendo os dados da coluna de uma tabela e outra contendo os dados de um valor. Depois, compara cada dado da coluna com esse valor, retornando a posição do dado valor na coluna em caso de sucesso (quando são iguais) e -1, caso contrário.

**Parameters**

|               |                                                   |
|---------------|---------------------------------------------------|
| <i>coluna</i> | Estrutura de texto contendo os dados da tabela.   |
| <i>valor</i>  | Estrutura de texto contendo os dados de um valor. |

**Returns**

-1 se o valor não estiver presente. Caso contrário, retorna a posição do valor na coluna.

#### 5.10.1.2 tabela\_adicionarLinha()

```
Tabela tabela_adicionarLinha (
    Tabela tabela,
    Texto dados[] )
```

Adiciona uma linha para cada coluna de uma tabela.

Recebe como parâmetro uma estrutura texto contendo os dados a serem adicionados e uma estrutura contendo os dados da tabela. Em seguida, atualiza a estrutura [Tabela](#) com os dados da estrutura [Texto](#) retornando-a no final.



## Parameters

|               |                                                                                  |
|---------------|----------------------------------------------------------------------------------|
| <i>tabela</i> | Estrutura de texto contendo os dados da tabela.                                  |
| <i>dados</i>  | Estrutura de string contendo os dados a serem adicionados nas colunas da tabela. |

## Returns

Estrutura [Tabela](#) contendo os dados da tabela atualizados.

### 5.10.1.3 `tabela_criar()`

```
Tabela tabela_criar (
    Texto nome,
    int quantidadeDeColunas,
    Texto nomesDasColunas[],
    char tiposDasColunas[] )
```

Cria uma tabela.

Recebe como argumentos o nome da tabela, a quantidade de colunas que a tabela terá, um vetor com os nomes das colunas e outro com os tipos de dados associados à elas. Retorna uma estrutura preenchida com essas informações.

## Parameters

|                            |                                                                   |
|----------------------------|-------------------------------------------------------------------|
| <i>nome</i>                | Estrutura contendo os dados do nome da tabela.                    |
| <i>quantidadeDeColunas</i> | A quantidade de colunas que a tabela terá.                        |
| <i>nomesDasColunas</i>     | Vetor de string contendo os nomes das colunas.                    |
| <i>tiposDasColunas</i>     | Vetor de string contendo os tipos de dados associados às colunas. |

## Returns

Estrutura preenchidas com as informações recebidas.

### 5.10.1.4 `tabela_exibirCabecalho()`

```
void tabela_exibirCabecalho (
    Tabela tabela )
```

Exibe na tela o cabeçalho de uma tabela.

Recebe como parâmetro uma estrutura contendo o nome de uma tabela, a quantidade de linhas e colunas, os nomes e os tipos das colunas e exibe essas informações na tela.

## Parameters

|               |                                                 |
|---------------|-------------------------------------------------|
| <i>tabela</i> | Estrutura de texto contendo os dados da tabela. |
|---------------|-------------------------------------------------|

### 5.10.1.5 `tabela_exibirDados()`

```
void tabela_exibirDados (
    Tabela tabela )
```

Exibe na tela os dados das colunas de uma tabela.

Recebe como parâmetro uma estrutura contendo os dados de uma tabela e exibe na tela os dados de cada coluna.

## Parameters

|               |                                                 |
|---------------|-------------------------------------------------|
| <i>tabela</i> | Estrutura de texto contendo os dados da tabela. |
|---------------|-------------------------------------------------|

#### 5.10.1.6 `tabela_gravar()`

```
void tabela_gravar (  
    Tabela tabela )
```

Cria um arquivo referente a uma tabela.

Recebe como argumento uma estrutura contendo informações de uma tabela adquiridas previamente em outra função e cria um arquivo para essa tabela, repassando as informações contidas na estrutura para esse arquivo.

Observação: o arquivo será nomeado pelo nome da tabela.

##### Parameters

|               |                                                  |
|---------------|--------------------------------------------------|
| <i>tabela</i> | Estrutura contendo as informações de uma tabela. |
|---------------|--------------------------------------------------|

#### 5.10.1.7 `tabela_liberar()`

```
void tabela_liberar (  
    Tabela tabela )
```

Liberação da memória alocada dinamicamente para a estrutura tabela.

##### Parameters

|               |                                                 |
|---------------|-------------------------------------------------|
| <i>tabela</i> | Estrutura de texto contendo os dados da tabela. |
|---------------|-------------------------------------------------|

#### 5.10.1.8 `tabela_recuperar()`

```
Tabela tabela_recuperar (  
    Texto nome )
```

Faz a leitura dos dados de um arquivo referente a uma tabela.

Recebe como argumento uma estrutura de texto contendo os dados do nome de uma tabela, abre o arquivo correspondente à essa tabela, em modo leitura e faz a leitura de seus dados, retornando uma estrutura contendo os dados lidos.

##### Parameters

|             |                                                         |
|-------------|---------------------------------------------------------|
| <i>nome</i> | Estrutura de texto contendo os dados do nome da tabela. |
|-------------|---------------------------------------------------------|

##### Returns

Uma estrutura contendo os dados lidos.

#### 5.10.1.9 `tabela_removeChave()`

```
int tabela_removeChave (  
    Tabela tabela,  
    Texto chave )
```

Remove uma linha (tupla) de uma tabela.

Recebe como parâmetros uma estrutura contendo os dados de uma tabela e outra contendo os dados de um valor correspondente à chave da linha a ser removida. Depois, localiza sua posição na coluna, caso ela exista e atualiza o arquivo da tabela com os dados restantes, excluindo-se os referentes à essa posição. Retorna 1 caso a remoção da linha seja realizada com sucesso e 0 caso a chave solicitada não exista.

##### Parameters

|               |                                                                                |
|---------------|--------------------------------------------------------------------------------|
| <i>tabela</i> | Estrutura de texto contendo os dados da tabela.                                |
| <i>chave</i>  | Estrutura de texto contendo os dados da chave referente à linha a ser removida |

**Returns**

1 caso a remoção da linha seja realizada com sucesso e 0 caso a chave não exista.

## 5.11 Tabela.h

[Go to the documentation of this file.](#)

```

00001
00002 #ifndef TABELA_H
00003 #define TABELA_H
00004
00005 #include <stdio.h>
00006 #include <string.h>
00007
00008 #include "Dados.h"
00009
00015 typedef struct {
00016     Texto nome;
00017     char tipo;
00018     int quantidadeDeLinhas;
00019     Texto * dados;
00020 } ColunaDaTabela;
00021
00027 typedef struct {
00028     Texto nome;
00029     int quantidadeDeColunas;
00030     int quantidadeDeLinhas;
00031     ColunaDaTabela * colunas;
00032 } Tabela;
00033
00034 Tabela tabela_criar(Texto nome, int quantidadeDeColunas, Texto nomeDasColunas[], char
    tipoDasColunas[]);
00035 void tabela_gravar(Tabela tabela);
00036 Tabela tabela_recuperar(Texto nome);
00037 void tabela_liberar(Tabela tabela);
00038 void tabela_exibirCabecalho(Tabela tabela);
00039 void tabela_exibirDados(Tabela tabela);
00040 int tabela_removeChave(Tabela tabela, Texto chave);
00041
00042 Tabela tabela_adicionarLinha(Tabela tabela, Texto dados []);
00043
00044 // retorna -1 se o valor não estiver na coluna, em caso contrário, retorna a posição em que ele se
    encontra
00045 int coluna_buscarValor(ColunaDaTabela coluna, Texto valor);
00046
00047 #endif

```

## 5.12 TabelaGeral.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include "Dados.h"
#include "TabelaGeral.h"

```

**Macros**

- #define NOME\_DA\_TABELA\_GERAL "tabela geral.data"

**Functions**

- Texto \* tabelaGeral\_recuperarDados (FILE \*arquivo, int \*n)
- void tabelaGeral\_gravarDados (FILE \*arquivo, int quantidadeDeTabelas, Texto nomesDasTabelas[])
- void tabelaGeral\_adicionarTabela (Texto nomeDaNovaTabela)
- int tabelaGeral\_contemTabela (Texto nome)
- void tabelaGeral\_limpar ()
- int tabelaGeral\_apagar (Texto excluindo)
- void tabelaGeral\_listar ()

## 5.12.1 Macro Definition Documentation

### 5.12.1.1 NOME\_DA\_TABELA\_GERAL

```
#define NOME_DA_TABELA_GERAL "tabela geral.data"
```

## 5.12.2 Function Documentation

### 5.12.2.1 tabelaGeral\_adicionarTabela()

```
void tabelaGeral_adicionarTabela (
    Texto nomeDaNovaTabela )
```

Cria ou atualiza um arquivo contendo a quantidade de tabelas e seus respectivos nomes, sempre que uma nova tabela é criada.

Caso o arquivo não exista, um novo é criado. Caso contrário, seus dados são atualizados.

#### Parameters

|                         |                                                         |
|-------------------------|---------------------------------------------------------|
| <i>nomeDaNovaTabela</i> | Estrutura com o nome da nova tabela criada previamente. |
|-------------------------|---------------------------------------------------------|

### 5.12.2.2 tabelaGeral\_apagar()

```
int tabelaGeral_apagar (
    Texto excluindo )
```

Apaga do disco rígido uma tabela específica e atualiza o arquivo que lista os nomes das tabelas existentes.

Recebe como argumento uma estrutura contendo o nome da tabela a ser apagada e atualiza os dados do arquivo que contém os nomes de todas as tabelas existentes, excluindo-se o nome da tabela que foi apagada.

#### Parameters

|                  |                                                |
|------------------|------------------------------------------------|
| <i>excluindo</i> | Estrutura contendo os dados a serem deletados. |
|------------------|------------------------------------------------|

#### Returns

1 se a tabela existir e a remoção for realizada e 0, caso a tabela não exista.

### 5.12.2.3 tabelaGeral\_contemTabela()

```
int tabelaGeral_contemTabela (
    Texto nome )
```

Recebe uma estrutura de texto que corresponde a um nome e verifica se esse nome já existe dentro de um arquivo. Abre um arquivo em modo leitura, realiza a leitura de seus dados e compara se o dado recebido é igual a algum dado lido. Retorna 1 em caso de sucesso e 0, caso contrário.

#### Parameters

|             |                                      |
|-------------|--------------------------------------|
| <i>nome</i> | Estrutura de <a href="#">Texto</a> . |
|-------------|--------------------------------------|

#### Returns

1 se o nome já existir e 0, caso contrário.

### 5.12.2.4 tabelaGeral\_gravarDados()

```
void tabelaGeral_gravarDados (
    FILE * arquivo,
```

```
int quantidadeDeTabelas,
Texto nomesDasTabelas[ ] )
```

Grava no arquivo os dados atualizados sobre a quantidade de tabelas e seus nomes.

#### Parameters

|                            |                                                                     |
|----------------------------|---------------------------------------------------------------------|
| <i>arquivo</i>             | Arquivo a ter dados gravados. Deve estar aberto no modo de escrita. |
| <i>quantidadeDeTabelas</i> | Quantidade atualizada.                                              |
| <i>nomesDasTabelas</i>     | <a href="#">Texto</a> com os dados atualizados.                     |

#### 5.12.2.5 tabelaGeral\_limpar()

```
void tabelaGeral_limpar ( )
```

Apaga do disco rígido todos os arquivos salvos.

#### 5.12.2.6 tabelaGeral\_listar()

```
void tabelaGeral_listar ( )
```

Lista os nomes das tabelas existentes.

Faz a leitura dos dados do arquivo 'tabela geral.data' e exibe na tela os dados correspondentes aos nomes das tabelas.

#### 5.12.2.7 tabelaGeral\_recuperarDados()

```
Texto * tabelaGeral_recuperarDados (
    FILE * arquivo,
    int * n )
```

Faz a leitura dos dados de um arquivo aberto em modo de leitura, atribuindo os dados lidos em uma estrutura de strings. Os dados do arquivo estão organizados na seguinte sequência: o primeiro dado corresponde à quantidade de tabelas existentes, enquanto que os dados restantes correspondem aos nomes das tabelas, representados pela estrutura [Texto](#).

#### Parameters

|                |                                      |
|----------------|--------------------------------------|
| <i>arquivo</i> | Arquivo a ser lido.                  |
| <i>n</i>       | Ponteiro para quantidade de tabelas. |

#### Returns

Uma estrutura de [Texto](#) contendo os dados lidos, além de passar por referência o dado correspondente à quantidade de tabelas.

## 5.13 TabelaGeral.h File Reference

```
#include <string.h>
#include "Dados.h"
#include "Tabela.h"
```

#### Functions

- int [tabelaGeral\\_contemTabela](#) ([Texto](#) nome)
- void [tabelaGeral\\_adicionarTabela](#) ([Texto](#) nome)
- void [tabelaGeral\\_limpar](#) ()
- int [tabelaGeral\\_apagar](#) ([Texto](#) nomeDaTabela)
- void [tabelaGeral\\_listar](#) ()

### 5.13.1 Function Documentation

#### 5.13.1.1 `tabelaGeral_adicionarTabela()`

```
void tabelaGeral_adicionarTabela (
    Texto nomeDaNovaTabela )
```

Cria ou atualiza um arquivo contendo a quantidade de tabelas e seus respectivos nomes, sempre que uma nova tabela é criada.

Caso o arquivo não exista, um novo é criado. Caso contrário, seus dados são atualizados.

##### Parameters

|                               |                                                         |
|-------------------------------|---------------------------------------------------------|
| <code>nomeDaNovaTabela</code> | Estrutura com o nome da nova tabela criada previamente. |
|-------------------------------|---------------------------------------------------------|

#### 5.13.1.2 `tabelaGeral_apagar()`

```
int tabelaGeral_apagar (
    Texto excluindo )
```

Apaga do disco rígido uma tabela específica e atualiza o arquivo que lista os nomes das tabelas existentes.

Recebe como argumento uma estrutura contendo o nome da tabela a ser apagada e atualiza os dados do arquivo que contém os nomes de todas as tabelas existentes, excluindo-se o nome da tabela que foi apagada.

##### Parameters

|                        |                                                |
|------------------------|------------------------------------------------|
| <code>excluindo</code> | Estrutura contendo os dados a serem deletados. |
|------------------------|------------------------------------------------|

##### Returns

1 se a tabela existir e a remoção for realizada e 0, caso a tabela não exista.

#### 5.13.1.3 `tabelaGeral_contemTabela()`

```
int tabelaGeral_contemTabela (
    Texto nome )
```

Recebe uma estrutura de texto que corresponde a um nome e verifica se esse nome já existe dentro de um arquivo. Abre um arquivo em modo leitura, realiza a leitura de seus dados e compara se o dado recebido é igual a algum dado lido. Retorna 1 em caso de sucesso e 0, caso contrário.

##### Parameters

|                   |                                   |
|-------------------|-----------------------------------|
| <code>nome</code> | Estrutura de <code>Texto</code> . |
|-------------------|-----------------------------------|

##### Returns

1 se o nome já existir e 0, caso contrário.

#### 5.13.1.4 `tabelaGeral_limpar()`

```
void tabelaGeral_limpar ( )
```

Apaga do disco rígido todos os arquivos salvos.

#### 5.13.1.5 `tabelaGeral_listar()`

```
void tabelaGeral_listar ( )
```

Lista os nomes das tabelas existentes.

Faz a leitura dos dados do arquivo 'tabela geral.data' e exibe na tela os dados correspondentes aos nomes das tabelas.

## 5.14 TabelaGeral.h

[Go to the documentation of this file.](#)

```
00001
00002 #ifndef TABELA_GERAL_H
00003 #define TABELA_GERAL_H
00004
00005 #include <string.h>
00006
00007 #include "Dados.h"
00008 #include "Tabela.h"
00009
00010 int tabelaGeral_contemTabela(Texto nome);
00011 void tabelaGeral_adicionarTabela(Texto nome);
00012 void tabelaGeral_limpar();
00013 int tabelaGeral_apagar(Texto nomeDaTabela);
00014 void tabelaGeral_listar();
00015
00016 #endif
```





# Index

- adicionarDados
  - principal.c, [21](#)
- APAGAR\_TABELA
  - principal.c, [20](#)
- APAGAR\_TUPLA
  - principal.c, [20](#)
- apagarTabela
  - principal.c, [21](#)
- apagarTupla
  - principal.c, [21](#)
- capacidade
  - Texto, [9](#)
- coluna\_buscarValor
  - Tabela.c, [23](#)
  - Tabela.h, [26](#)
- ColunaDaTabela, [7](#)
  - dados, [7](#)
  - nome, [7](#)
  - quantidadeDeLinhas, [7](#)
  - tipo, [7](#)
- colunas
  - Tabela, [8](#)
- compararProximo
  - Pesquisa.c, [17](#)
- compararValores
  - Pesquisa.c, [18](#)
- CRIAR\_LINHA
  - principal.c, [20](#)
- CRIAR\_TABELA
  - principal.c, [20](#)
- criarTabela
  - principal.c, [21](#)
- dados
  - ColunaDaTabela, [7](#)
- Dados.c, [11](#)
  - ehDigito, [11](#)
  - ehMaiuscula, [11](#)
  - ehMinuscula, [12](#)
  - ehSimbolo, [12](#)
  - texto\_gravar, [12](#)
  - texto\_lerTerminalEmLoop, [12](#)
  - texto\_lerTipoEmLoop, [13](#)
  - texto\_lerValorComTipo, [13](#)
  - texto\_lerValorDoTipoString, [13](#)
  - texto\_lerValorPositivoEmLoop, [13](#)
  - texto\_recuperar, [14](#)
  - verificaNomeDaTabelaValido, [14](#)
- Dados.h, [14](#)
- texto\_gravar, [15](#)
- texto\_lerTerminalEmLoop, [15](#)
- texto\_lerTipoEmLoop, [15](#)
- texto\_lerValorComTipo, [15](#)
- texto\_lerValorPositivoEmLoop, [16](#)
- texto\_recuperar, [16](#)
- deveExibir
  - Pesquisa.c, [18](#)
- ehDigito
  - Dados.c, [11](#)
- ehMaiuscula
  - Dados.c, [11](#)
- ehMinuscula
  - Dados.c, [12](#)
- ehSimbolo
  - Dados.c, [12](#)
- exibirDados
  - principal.c, [21](#)
- IGUAL
  - Pesquisa.c, [17](#)
- leituraDaChavePrimaria
  - principal.c, [21](#)
- leituraDeDados
  - principal.c, [21](#)
- LIMPAR
  - principal.c, [20](#)
- LISTAR\_DADOS
  - principal.c, [20](#)
- LISTAR\_TABELA
  - principal.c, [20](#)
- main
  - principal.c, [22](#)
- MAIOR
  - Pesquisa.c, [17](#)
- MAIOR\_OU\_IGUAL
  - Pesquisa.c, [17](#)
- MENOR
  - Pesquisa.c, [17](#)
- MENOR\_OU\_IGUAL
  - Pesquisa.c, [17](#)
- menu
  - principal.c, [22](#)
- nome
  - ColunaDaTabela, [7](#)
  - Tabela, [8](#)
- NOME\_DA\_TABELA\_GERAL

- TabelaGeral.c, 30
- Opcoes
  - principal.c, 20
- Pesquisa.c, 17
  - compararProximo, 17
  - compararValores, 18
  - deveExibir, 18
  - IGUAL, 17
  - MAIOR, 17
  - MAIOR\_OU\_IGUAL, 17
  - MENOR, 17
  - MENOR\_OU\_IGUAL, 17
  - pesquisa\_exibir, 18
  - pesquisar, 19
  - PROXIMO, 17
  - Regra, 17
- Pesquisa.h, 19
  - pesquisar, 19
- pesquisa\_exibir
  - Pesquisa.c, 18
- pesquisar
  - Pesquisa.c, 19
  - Pesquisa.h, 19
- PESQUISAR\_VALOR
  - principal.c, 20
- pesquisarValores
  - principal.c, 22
- principal.c, 20
  - adicionarDados, 21
  - APAGAR\_TABELA, 20
  - APAGAR\_TUPLA, 20
  - apagarTabela, 21
  - apagarTupla, 21
  - CRIAR\_LINHA, 20
  - CRIAR\_TABELA, 20
  - criarTabela, 21
  - exibirDados, 21
  - leituraDaChavePrimaria, 21
  - leituraDeDados, 21
  - LIMPAR, 20
  - LISTAR\_DADOS, 20
  - LISTAR\_TABELA, 20
  - main, 22
  - menu, 22
  - Opcoes, 20
  - PESQUISAR\_VALOR, 20
  - pesquisarValores, 22
  - SAIR, 20
  - verificarExistenciaDaChave, 22
  - verificarNomesIguais, 22
- PROXIMO
  - Pesquisa.c, 17
- quantidadeDeColunas
  - Tabela, 8
- quantidadeDeLinhas
  - ColunaDaTabela, 7
- Tabela, 8
  - colunas, 8
  - nome, 8
  - quantidadeDeColunas, 8
  - quantidadeDeLinhas, 8
- README.md, 23
- Regra
  - Pesquisa.c, 17
- SAIR
  - principal.c, 20
- SGBD ITP, 1
- Tabela, 8
  - colunas, 8
  - nome, 8
  - quantidadeDeColunas, 8
  - quantidadeDeLinhas, 8
- Tabela.c, 23
  - coluna\_buscarValor, 23
  - tabela\_adicionarLinha, 23
  - tabela\_criar, 24
  - tabela\_exibirCabecalho, 24
  - tabela\_exibirDados, 24
  - tabela\_gravar, 24
  - tabela\_liberar, 25
  - tabela\_recuperar, 25
  - tabela\_removeChave, 25
- Tabela.h, 26
  - coluna\_buscarValor, 26
  - tabela\_adicionarLinha, 26
  - tabela\_criar, 27
  - tabela\_exibirCabecalho, 27
  - tabela\_exibirDados, 27
  - tabela\_gravar, 28
  - tabela\_liberar, 28
  - tabela\_recuperar, 28
  - tabela\_removeChave, 28
- tabela\_adicionarLinha
  - Tabela.c, 23
  - Tabela.h, 26
- tabela\_criar
  - Tabela.c, 24
  - Tabela.h, 27
- tabela\_exibirCabecalho
  - Tabela.c, 24
  - Tabela.h, 27
- tabela\_exibirDados
  - Tabela.c, 24
  - Tabela.h, 27
- tabela\_gravar
  - Tabela.c, 24
  - Tabela.h, 28
- tabela\_liberar
  - Tabela.c, 25
  - Tabela.h, 28
- tabela\_recuperar
  - Tabela.c, 25
  - Tabela.h, 28
- tabela\_removeChave
  - Tabela.c, 25
  - Tabela.h, 28

- TabelaGeral.c, [29](#)
  - NOME\_DA\_TABELA\_GERAL, [30](#)
  - tabelaGeral\_adicionarTabela, [30](#)
  - tabelaGeral\_apagar, [30](#)
  - tabelaGeral\_contemTabela, [30](#)
  - tabelaGeral\_gravarDados, [30](#)
  - tabelaGeral\_limpar, [31](#)
  - tabelaGeral\_listar, [31](#)
  - tabelaGeral\_recuperarDados, [31](#)
- TabelaGeral.h, [31](#)
  - tabelaGeral\_adicionarTabela, [32](#)
  - tabelaGeral\_apagar, [32](#)
  - tabelaGeral\_contemTabela, [32](#)
  - tabelaGeral\_limpar, [32](#)
  - tabelaGeral\_listar, [32](#)
- tabelaGeral\_adicionarTabela
  - TabelaGeral.c, [30](#)
  - TabelaGeral.h, [32](#)
- tabelaGeral\_apagar
  - TabelaGeral.c, [30](#)
  - TabelaGeral.h, [32](#)
- tabelaGeral\_contemTabela
  - TabelaGeral.c, [30](#)
  - TabelaGeral.h, [32](#)
- tabelaGeral\_gravarDados
  - TabelaGeral.c, [30](#)
- tabelaGeral\_limpar
  - TabelaGeral.c, [31](#)
  - TabelaGeral.h, [32](#)
- tabelaGeral\_listar
  - TabelaGeral.c, [31](#)
  - TabelaGeral.h, [32](#)
- tabelaGeral\_recuperarDados
  - TabelaGeral.c, [31](#)
- tamanho
  - Texto, [9](#)
- Texto, [8](#)
  - capacidade, [9](#)
  - tamanho, [9](#)
  - vetor, [9](#)
- texto\_gravar
  - Dados.c, [12](#)
  - Dados.h, [15](#)
- texto\_lerTerminalEmLoop
  - Dados.c, [12](#)
  - Dados.h, [15](#)
- texto\_lerTipoEmLoop
  - Dados.c, [13](#)
  - Dados.h, [15](#)
- texto\_lerValorComTipo
  - Dados.c, [13](#)
  - Dados.h, [15](#)
- texto\_lerValorDoTipoString
  - Dados.c, [13](#)
- texto\_lerValorPositivoEmLoop
  - Dados.c, [13](#)
  - Dados.h, [16](#)
- texto\_recuperar
  - Dados.c, [14](#)
  - Dados.h, [16](#)
- tipo
  - ColunaDaTabela, [7](#)
- verificaNomeDaTabelaValido
  - Dados.c, [14](#)
- verificarExistenciaDaChave
  - principal.c, [22](#)
- verificarNomesIguais
  - principal.c, [22](#)
- vetor
  - Texto, [9](#)